

1. SYSTEM DESIGN & ARCHITECTURE

Question: "Can you walk me through the architecture of SpeakMirror and explain the rationale behind your design decisions?"

Narrative Storytelling Response (STAR Method):

- [Situation]** When planning SpeakMirror, streaming high-definition video feeds to a cloud server to analyze speech introduced severe network latency (often 4-6 seconds) and incurred unsustainable server GPU hosting costs.
- [Task]** I had to architect a zero-latency, cost-effective system that could analyze eye contact and facial expressions in real time, while still utilizing advanced language models for transcript feedback.
- [Action]** I split the architecture: client-side visual telemetry runs locally using Google's MediaPipe FaceLandmarker WebAssembly (WASM) directly on the client's GPU at 60 FPS. Meanwhile, serverless vocal diagnostics are routed at recording completion to Groq Whisper-large-v3 for transcription and Llama 3.1 LLM to generate unified visual-vocal feedback.
- [Result]** This edge-vocal architecture resulted in zero video-network latency, dropped server GPU hosting requirements to zero, and unified visual and audio analytics into cohesive coaching insights.

2. PERFORMANCE & OPTIMIZATION

Question: "You ran a 3D FaceLandmarker model in the browser at 60 FPS. How did you optimize React to prevent UI lag and browser lockups?"

Narrative Storytelling Response (STAR Method):

- [Situation]** In the initial prototype, triggering standard React state updates inside the animation frame loop to update the live HUD meters caused 60 re-renders per second, freezing the video recording preview and causing high CPU utilization.
- [Task]** I needed to decouple the high-frequency WASM processing loop from React's rendering pipeline to achieve a smooth 60 FPS video recording preview.
- [Action]** I stored the landmarker instances, frame IDs, and tracking arrays inside refs rather than React states, running the computer vision loop silently. I then implemented a timestamp check (`performance.now()`) that throttled React state updates to once every 150ms (reducing re-renders from 60 to ~7 per second). Finally, I attached cleanup listeners to free WebAssembly memory on unmount.
- [Result]** CPU usage dropped by over 75%, page lag was completely eliminated, and we achieved a smooth 60 FPS video recording preview on desktop and mobile.

3. CROSS-BROWSER COMPATIBILITY

Question: "How did you solve the cross-browser recording and playback compatibility issues (specifically regarding WebM vs MP4 containers)?"

Narrative Storytelling Response (STAR Method):

[Situation]	Browser media recording is highly fragmented. Safari only supports MP4 (H.264/AAC) and crashes with WebM, while Firefox and older Chrome versions only support WebM, which fails to play natively on standard slide presentations, iOS QuickTime, Slack, or WhatsApp.
[Task]	I had to build a recording pipeline that works on Chrome, Safari, and Firefox, and outputs files in a format that can be played universally.
[Action]	I wrote a runtime feature-negotiation module using <code>MediaRecorder.isTypeSupported()</code> to check and select MP4 (H.264/AAC) containers first, falling back to WebM container types. I then updated the Supabase file storage upload script to dynamically extract the extension (mp4/webm) from the recorded Blob type, and refactored the download handlers to fetch the correct file headers.
[Result]	The system achieves 100% browser compatibility. Safari users record in native MP4, Chrome/Edge users record in MP4 when supported, and Firefox users fall back to WebM. All exported files play seamlessly on all platforms.

4. EDGE ML & MATHEMATICAL MODELING

Question: "How did you calculate 'Eye Contact Quality' and 'Facial Expression Score' from raw facial blendshapes?"

Narrative Storytelling Response (STAR Method):

[Situation]	MediaPipe's FaceLandmarker returns a raw list of 52 blendshape values (decimal scores between 0.0 and 1.0) which do not represent human-friendly indicators for gaze focus or facial engagement.
[Task]	I needed to design mathematical models for 'Eye Contact Quality' and 'Facial Expression' that align with human coaching perception and allow for natural movement.
[Action]	I calculated horizontal eye gaze shift (gazeX) by comparing inward and outward look blendshapes, and vertical shift (gazeY) by comparing lookUp and lookDown scores. Using the Euclidean distance deviation, I established a dead-zone of 0.08 (allowing natural micro-movements) and a linear drop-off threshold up to 0.28 (looking away). For expressions, I averaged smile intensity (40%), eyebrow raising (30%), and eye widening (30%) to score engagement rather than just smiles.
[Result]	The metrics feel natural. Users can look around the screen without penalization, but looking fully away drops eye contact immediately. The expression score rewards visual warmth and facial animation.

5. DATABASE SECURITY & TENANCY

Question: "How did you design the database schemas and security policies to handle sandboxed personal practices vs shared team workspaces?"

Narrative Storytelling Response (STAR Method):

[Situation]	SpeakMirror caters to B2C (private sandboxes) and B2B (collaborative team organizations). A major risk in PostgreSQL RLS (Row-Level Security) is recursive query loops that slow down data queries and cause memory exhaustion under load.
[Task]	I had to write high-performance, recursion-free Postgres policies that strictly separate personal workspace records from team records.
[Action]	I mapped all rooms, tasks, and recordings to an <code>organization_id</code> . Every user upon signup is provisioned a 'Personal Space' organization. I then wrote a recursion-free PL/pgSQL helper function marked as 'security definer' to query memberships directly using the session context (bypassing table loops), and mapped this function to all recordings RLS policies.
[Result]	The database enforces absolute multi-tenancy. Personal recordings are fully private, while team workflows allow collaborative mentor reviews with minimal query latency.

